Funktionsreferenz PCB_test_code

Kurzübersicht aller Funktionen der Test-PCB-Firmware (STM32F103RB, ADS8681, AD5689R) · Übergabe-Doku · Stand 22.10.2026

Alle Dateien liegen in diesem Ordner (Keil-Projekt `PCB_test.uvprojx`). Ausführlichere Erklärungen: `Dokumentation/Firmware_Doku_PCB_test_code.pdf` und `Dokumentation/Messroutinen_IDVG_Transient.pdf`.

main.c

Funktion	Was sie macht	Verwendung
`int main(void)`	Initialisiert Pins, UART, SPI, Versorgungen, Relais (100 kΩ, Eingang GND), DACs auf 0 V, setzt Standardwerte und läuft dann in der Hauptschleife.	Einstiegspunkt. Hauptschleife: Befehl decodieren, dann je nach Flag IDVG / transient / stress / wait_temp_equi.

PINs.c / PINs.h

Funktion	Was sie macht	Verwendung
`void PIN_init(void)`	Schaltet Takt für GPIOA/B/C ein und konfiguriert alle Pins (CS, Relais, PSU-Enable, LEDs, SPI-Pins PA5–PA7).	`Einmal als Erstes in main().`
`Makros ADC_SPI_NCS, DAC_SPI_NCS, LED_RED, …`	Bit-Banding-Zugriff auf einzelne Pins.	`LED_GREEN = 1; ADC_SPI_NCS = 0;`

clock.c

Funktion	Was sie macht	Verwendung
`void clock_init(void)`	APB1/APB2-Teiler = 1, PLL aus → Chip läuft auf internem 8-MHz-Takt.	`Wird derzeit nicht aufgerufen (Standard ist ohnehin HSI 8 MHz).`

NVIC.c

Funktion	Was sie macht	Verwendung
`void NVIC_init(char position, char priority)`	Setzt Priorität eines Interrupts, löscht Pending-Bit, gibt ihn frei.	`NVIC_init(USART1_IRQn, 2);`

UART.c

Funktion	Was sie macht	Verwendung
`void set_up_uart1(void)`	Konfiguriert PA9 (TX) / PA10 (RX), USART1 8N1, 9600 Baud, NVIC-Priorität 2.	Einmal am Start.
`void uart1_Rx_Interupt(bool setting)`	Schaltet den Empfangs-Interrupt (RXNEIE) ein/aus.	`uart1_Rx_Interupt(true);` ← muss nach `set_up_uart1()` aufgerufen werden!
`void uart1_set_baud(uint32_t baud)`	Setzt Baudrate aus SystemCoreClock.	`uart1_set_baud(115200);`
`void uart_put_char(char c)`	Sendet ein Zeichen (blockierend).	`uart_put_char('A');`
`void uart_put_string(char *s)`	Sendet einen String (blockierend).	`uart_put_string("END\r\n");`
`bool uart_string_received(void)`	true, wenn eine komplette Zeile empfangen wurde.	`if (uart_string_received()) …`
`char* uart_get_string(void)`	Gibt die empfangene Zeile zurück und gibt den Puffer frei (NULL wenn nichts da).	`get_Decode_Parameter(uart_get_string(), &measure);`
`void USART1_IRQHandler(void)`	Interrupt: sammelt Zeichen bis \r/\n; erkennt „ABOR“/„ABORT“ immer und setzt abort_requested.	Nicht selbst aufrufen.
`volatile bool abort_requested`	Abbruch-Flag.	In Schleifen abfragen, am Messbeginn auf false setzen.

SPI.c

CS (ADC_SPI_NCS / DAC_SPI_NCS) muss der Aufrufer selbst setzen.

Funktion	Was sie macht	Verwendung
`void spi1_init(void)`	SPI1 Master, 8 Bit, CPOL 0 / CPHA 1, Software-NSS, RX-DMA an.	Einmal am Start.
`void spi1_set_baud(uint8_t br)`	Setzt SPI-Teiler (0 = /2 … 7 = /256).	`spi1_set_baud(5); // /64`
`uint8_t spi1_transfer8(uint8_t tx)`	Sendet 1 Byte, gibt empfangenes Byte zurück.	`rx = spi1_transfer8(0x00);`
`uint16_t spi1_transfer16(uint16_t tx)`	Sendet/empfängt 2 Bytes (MSB zuerst).	`rx = spi1_transfer16(0x0000);`

dma.c

Funktion	Was sie macht	Verwendung
`void spi1_rx_dma_init(void)`	DMA1 Kanal 2: SPI1-DR → Speicher, 8 Bit.	`Einmal vor spi1_rx_dma().`
`void spi1_rx_dma(uint8_t *buf, uint16_t n)`	Startet Empfang von n Bytes in buf.	`spi1_rx_dma(buffer, 4000);`
`void spi1_tx_dma_init(void)`	DMA1 Kanal 3: Speicher → SPI1-DR, 8 Bit.	`Einmal vor spi1_tx_dma().`
`void spi1_tx_dma(uint8_t *buf, uint16_t n)`	Startet Senden von n Bytes.	`spi1_tx_dma(dummy, 2);`

TIMER.c

Funktion	Was sie macht	Verwendung
`void wait_ms(int ms)`	Warteschleife (auf 8 MHz abgestimmt).	`wait_ms(100);`
`bool delay_ms_abortable(uint32_t ms)`	Wartet ms, bricht bei ABORT ab. Rückgabe true = abgebrochen.	`if (delay_ms_abortable(500)) break;`
`void init_tim2(uint32_t freq)`	TIM2 auf Frequenz freq [Hz], Update-Interrupt an.	`init_tim2(1000);`
`void tim2_enable(void) / tim2_disable(void)`	Startet / stoppt TIM2.	`tim2_enable();`
`void TIM2_IRQHandler(void)`	Pro Timer-Tick: CS low, 2 Dummy-Bytes per DMA senden (→ ADC-Sample), CS high.	Nicht selbst aufrufen.

ads8681.c (ADC)

ADC_FSR_HALF in ads8681.c muss zum eingestellten ADC-Bereich passen.

Funktion	Was sie macht	Verwendung
`void ADS8681_init(void)`	Hardware-Reset über ADC_NRESET, ein NOP-Frame. (Range-Setzen auskommentiert → ±3·VREF.)	`Einmal am Start (derzeit nicht in main()!).`
`uint32_t ADS8681_transfer_frame(uint32_t tx)`	Ein 32-Bit-Frame mit CS. Obere 16 Bit der Antwort = letzter Messwert.	`raw = ADS8681_transfer_frame(0) >> 16;`
`uint32_t ADS8681_send_Command_blocking(cmd, reg, data)`	Befehl an ein Register senden, mit NOP-Frame Antwort lesen.	`…(ADS868X_SPI_COMMAND_WRITE_FULL, ADS868X_REGISTER_ADDRESS_RANGE_SEL, 0x0004);`
`double ADS8681_get_voltage(uint16_t raw)`	Rohwert → Spannung [V] (0x8000 = 0 V, Bereich ±3·VREF).	`u = ADS8681_get_voltage(raw);`
`double ADS8681_get_current(uint16_t raw)`	Rohwert → Strom [µA]: I = 3·U/Rf, Rf aus range_100n.	`i = ADS8681_get_current(raw);`

AD5689.c (DAC)

Adressen: AD5689_ADDR_DAC_A, AD5689_ADDR_DAC_B, AD5689_ADDR_DAC_AB. Verstärkung der DAC-Treiberstufe ist nicht eingerechnet.

Funktion	Was sie macht	Verwendung
`void AD5689_init(void)`	Hardware-Reset, Gain = 1, Power-up, beide Kanäle 0.	`Einmal am Start (derzeit nicht in main()!).`
`void AD5689_send_command(cmd, addr, data)`	24-Bit-Frame: 4 Bit Befehl, 4 Bit Adresse, 16 Bit Daten.	`AD5689_send_command(AD5689_CMD_WRITE_DAC, AD5689_ADDR_DAC_A, 0x8000);`
`void AD5689_set_Voltage(float V, addr)`	Spannung 0…2,5 V setzen (wird begrenzt).	`AD5689_set_Voltage(1.2f, AD5689_ADDR_DAC_A);`

relai.c

Bistabile Relais: jede Funktion gibt einen 100-ms-Puls und blockiert so lange.

Funktion	Was sie macht	Verwendung
`void set_range_10meg(void)`	Relais auf Rf = 10 MΩ (100-nA-Bereich), range_100n = true.	`set_range_10meg();`
`void set_range_100k(void)`	Relais auf Rf = 100 kΩ (10-µA-Bereich), range_100n = false.	`set_range_100k();`
`void set_GND_Relai(void)`	TIA-Eingang auf GND (Schutz, z. B. beim Stressen/HCD).	`set_GND_Relai();`
`void reset_GND_Relai(void)`	TIA-Eingang wieder zum DUT.	`reset_GND_Relai();`

COMMAND_DECODE.c

Befehlsformat: NAME= wert\n (Leerzeichen nach =). Liste aller Befehle in SCPI_COMMANDS.h und in der Firmware-Doku.

Funktion	Was sie macht	Verwendung
`void get_Decode_Parameter(char *cmd, Measure_Param *p)`	Vergleicht den Befehl mit allen bekannten Befehlen und schreibt den Wert ins passende Feld der Struct. Unbekannte Befehle werden ignoriert.	`get_Decode_Parameter(uart_get_string(), &measure);`
`Measure_Param (Struct, .h)`	Alle Parameter: source1_* (Gate), source2_* (Drain), sense_*, stress_*, temp_equilibrium_wait.	`Eine Instanz in main(), per Zeiger weitergeben.`

SCPI_COMMANDS.c

Funktion	Was sie macht	Verwendung
`void uart1_init / uart1_put_char / uart1_put_string`	Wrapper um die UART.c-Funktionen.	`uart1_put_string("OK\r\n");`
`void uart1_put_int(int v)`	Ganzzahl als Text senden.	`uart1_put_int(42);`
`void uart1_put_float(float v, int dec)`	Kommazahl mit dec Nachkommastellen senden (ohne printf).	`uart1_put_float(1.25f, 3);`
`scpi_reset, scpi_set_source_*, scpi_set_sense_*, scpi_start/stop_measurement`	Senden SCPI-Textbefehle über UART (z. B. „SOURce1:VOLTage 1.00“).	`Für Ausgabe/Test; das Gerät selbst wertet das andere Format (NAME= wert) aus.`

measure.c

Wichtig: jede Funktion muss am Ende ihr Flag (z. B. p->sense_mode_idvg = false) zurücksetzen und Gate/Drain auf 0 V bringen, sonst startet sie in der Hauptschleife endlos neu.

Funktion	Was sie macht	Verwendung
`void IDVG(Measure_Param *p)`	ID-VG-Sweep – **leer, noch zu implementieren**.	`Aufruf aus main() wenn sense_mode_idvg.`
`void transient(Measure_Param *p)`	Strom über Zeit bei fixem Gate – **leer**.	… wenn sense_mode_transient.
`void stress(Measure_Param *p)`	Stress BTI/HCD, statisch/dynamisch – **leer**.	… wenn stress_output.
`void wait_temp_equi(Measure_Param *p)`	Warten auf thermisches Gleichgewicht – **leer**.	… wenn temp_equilibrium_wait.
`void start_measure(Measure_Param *p)`	Alte Auswahl über sweep_step (wird von main() nicht mehr verwendet).	–

Offen für die Weiterarbeit

- `uart1_Rx_Interupt(true);` nach `set_up_uart1()` in main() einfügen – sonst kommen keine Befehle an.
- `ADS8681_init()` und `AD5689_init()` in main() aufrufen.
- `temp.c / temp.h` sind leer (Temperatursensor).
- Messroutinen in measure.c schreiben (Vorlage: Dokumentation/Messroutinen_IDVG_Transient.pdf).

Quellen

- ADS8681 Datenblatt: https://www.ti.com/lit/ds/symlink/ads8681.pdf
- AD5689R Datenblatt: https://www.analog.com/media/en/technical-documentation/data-sheets/AD5689R_5687R.pdf
- STM32F10x Reference Manual RM0008: https://www.st.com/resource/en/reference_manual/rm0008-stm32f101xx-stm32f102xx-stm32f103xx-stm32f105xx-and-stm32f107xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf